

Stocks: Logistic regression

Trevor Hastie and Robert Tibshirani

Here, I am adapting part of the lab associated with Chapter 4 of the textbook.

The Stock Market Data

We will begin by examining some numerical and graphical summaries of the `Smarket` data, which is part of the `ISLR2` library. This data set consists of percentage returns for the S&P 500 stock index over 1,250 days, from the beginning of 2001 until the end of 2005. For each date, we have recorded the percentage returns for each of the five previous trading days, `lagone` through `lagfive`. We have also recorded `volume` (the number of shares traded on the previous day, in billions), `Today` (the percentage return on the date in question) and `direction` (whether the market was Up or Down on this date). Our goal is to predict `direction` (a qualitative response) using the other features.

```
library(ISLR2)
names(Smarket)
```

```
## [1] "Year"      "Lag1"      "Lag2"      "Lag3"      "Lag4"      "Lag5"
## [7] "Volume"    "Today"     "Direction"
```

```
dim(Smarket)
```

```
## [1] 1250    9
```

```
summary(Smarket)
```

```
##      Year      Lag1      Lag2      Lag3
## Min.   :2001   Min.   :-4.922000   Min.   :-4.922000   Min.   :-4.922000
## 1st Qu.:2002   1st Qu.: -0.639500   1st Qu.: -0.639500   1st Qu.: -0.640000
## Median :2003   Median : 0.039000   Median : 0.039000   Median : 0.038500
## Mean   :2003   Mean   : 0.003834   Mean   : 0.003919   Mean   : 0.001716
## 3rd Qu.:2004   3rd Qu.: 0.596750   3rd Qu.: 0.596750   3rd Qu.: 0.596750
## Max.   :2005   Max.   : 5.733000   Max.   : 5.733000   Max.   : 5.733000
##      Lag4      Lag5      Volume      Today
## Min.   :-4.922000   Min.   :-4.922000   Min.   :0.3561   Min.   :-4.922000
## 1st Qu.: -0.640000   1st Qu.: -0.640000   1st Qu.:1.2574   1st Qu.: -0.639500
## Median : 0.038500   Median : 0.038500   Median :1.4229   Median : 0.038500
## Mean   : 0.001636   Mean   : 0.00561    Mean   :1.4783   Mean   : 0.003138
## 3rd Qu.: 0.596750   3rd Qu.: 0.59700    3rd Qu.:1.6417   3rd Qu.: 0.596750
## Max.   : 5.733000   Max.   : 5.73300    Max.   :3.1525   Max.   : 5.733000
## Direction
## Down:602
## Up  :648
##
##
##
##
```

```
#pairs(Smarket)
```

The `cor()` function produces a matrix that contains all of the pairwise correlations among the predictors in a data set. The first command below gives an error message because the `direction` variable is qualitative.

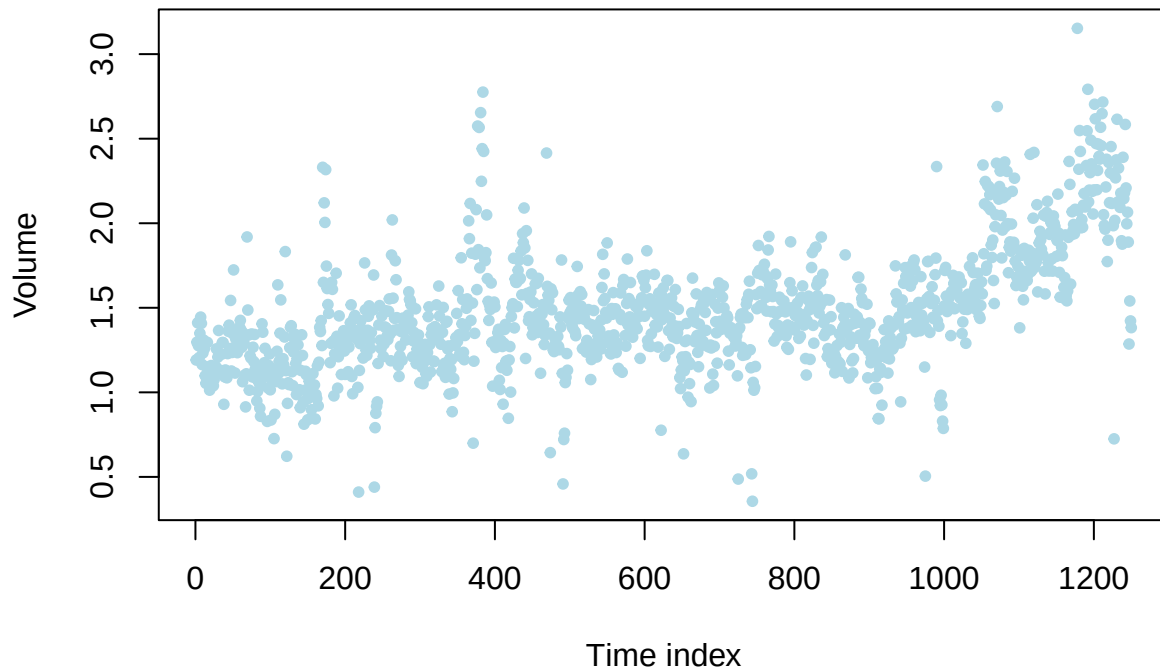
```
#cor(Smarket)
cor(Smarket[, -9])
```

```
##           Year      Lag1      Lag2      Lag3      Lag4
## Year  1.00000000  0.029699649  0.030596422  0.033194581  0.035688718
## Lag1  0.02969965  1.000000000 -0.026294328 -0.010803402 -0.002985911
## Lag2  0.03059642 -0.026294328  1.000000000 -0.025896670 -0.010853533
## Lag3  0.03319458 -0.010803402 -0.025896670  1.000000000 -0.024051036
## Lag4  0.03568872 -0.002985911 -0.010853533 -0.024051036  1.000000000
## Lag5  0.02978799 -0.005674606 -0.003557949 -0.018808338 -0.027083641
## Volume 0.53900647  0.040909908 -0.043383215 -0.041823686 -0.048414246
## Today 0.03009523 -0.026155045 -0.010250033 -0.002447647 -0.006899527
##           Lag5      Volume      Today
## Year  0.029787995  0.53900647  0.030095229
## Lag1 -0.005674606  0.04090991 -0.026155045
## Lag2 -0.003557949 -0.04338321 -0.010250033
## Lag3 -0.018808338 -0.04182369 -0.002447647
## Lag4 -0.027083641 -0.04841425 -0.006899527
## Lag5  1.000000000 -0.02200231 -0.034860083
## Volume -0.022002315  1.00000000  0.014591823
## Today -0.034860083  0.01459182  1.000000000
```

As one would expect, the correlations between the lag variables and today's returns are close to zero. In other words, there appears to be little correlation between today's returns and previous days' returns. The only substantial correlation is between `Year` and `Volume`. By plotting the data, which is ordered chronologically, we see that `Volume` is increasing over time. In other words, the average number of shares traded daily increased from 2001 to 2005.

```
attach(Smarket)
plot(Volume,
     pch=20, col="lightblue",
     main="Trade volume",
     xlab="Time index", ylab="Volume")
```

Trade volume



Logistic Regression

Next, we will fit a logistic regression model in order to predict `direction` using `lagone` through `lagfive` and `volume`. The `glm()` function can be used to fit many types of generalized linear models, including logistic regression. The syntax of the `glm()` function is similar to that of `lm()`, except that we must pass in the argument `family = binomial` in order to tell R to run a logistic regression rather than some other type of generalized linear model.

```
glm.fits <- glm(
  Direction ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5 + Volume,
  data = Smarket, family = binomial
)
summary(glm.fits)
```

```
##
## Call:
## glm(formula = Direction ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5 +
##      Volume, family = binomial, data = Smarket)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept) -0.126000   0.240736  -0.523   0.601
## Lag1        -0.073074   0.050167  -1.457   0.145
## Lag2        -0.042301   0.050086  -0.845   0.398
## Lag3         0.011085   0.049939   0.222   0.824
## Lag4         0.009359   0.049974   0.187   0.851
## Lag5         0.010313   0.049511   0.208   0.835
## Volume       0.135441   0.158360   0.855   0.392
##
## (Dispersion parameter for binomial family taken to be 1)
```

```
##
## Null deviance: 1731.2 on 1249 degrees of freedom
## Residual deviance: 1727.6 on 1243 degrees of freedom
## AIC: 1741.6
##
## Number of Fisher Scoring iterations: 3
```

The smallest p -value here is associated with `lagone`. The negative coefficient for this predictor suggests that if the market had a positive return yesterday, then it is less likely to go up today. However, at a value of 0.15, the p -value is still relatively large, and so there is no clear evidence of a real association between `lagone` and direction.

We use the `coef()` function in order to access just the coefficients for this fitted model. We can also use the `summary()` function to access particular aspects of the fitted model, such as the p -values for the coefficients.

```
#just the coefficients
coef(glm.fits)
```

```
## (Intercept)      Lag1      Lag2      Lag3      Lag4      Lag5
## -0.126000257 -0.073073746 -0.042301344  0.011085108  0.009358938  0.010313068
## Volume
## 0.135440659
```

```
#characteristics of coefficients
summary(glm.fits)$coef
```

```
## Estimate Std. Error z value Pr(>|z|)
## (Intercept) -0.126000257 0.24073574 -0.5233966 0.6006983
## Lag1 -0.073073746 0.05016739 -1.4565986 0.1452272
## Lag2 -0.042301344 0.05008605 -0.8445733 0.3983491
## Lag3 0.011085108 0.04993854 0.2219750 0.8243333
## Lag4 0.009358938 0.04997413 0.1872757 0.8514445
## Lag5 0.010313068 0.04951146 0.2082966 0.8349974
## Volume 0.135440659 0.15835970 0.8552723 0.3924004
```

```
#just the p-values
```

```
summary(glm.fits)$coef[, 4]
```

```
## (Intercept)      Lag1      Lag2      Lag3      Lag4      Lag5
## 0.6006983 0.1452272 0.3983491 0.8243333 0.8514445 0.8349974
## Volume
## 0.3924004
```

The `predict()` function can be used to predict the probability that the market will go up, given values of the predictors. The `type = "response"` option tells R to output probabilities of the form $P(Y = 1|X)$, as opposed to other information such as the logit. If no data set is supplied to the `predict()` function, then the probabilities are computed for the training data that was used to fit the logistic regression model. Here we have printed only the first ten probabilities. We know that these values correspond to the probability of the market going up, rather than down, because the `contrasts()` function indicates that R has created a dummy variable with a 1 for Up.

```
glm.probs <- predict(glm.fits, type = "response")
glm.probs[1:10]
```

```
## 1 2 3 4 5 6 7 8
## 0.5070841 0.4814679 0.4811388 0.5152224 0.5107812 0.5069565 0.4926509 0.5092292
## 9 10
## 0.5176135 0.4888378
```

```
contrasts(Direction)
```

```
##      Up
## Down  0
## Up    1
```

In order to make a prediction as to whether the market will go up or down on a particular day, we must convert these predicted probabilities into class labels, `Up` or `Down`. The following two commands create a vector of class predictions based on whether the predicted probability of a market increase is greater than or less than 0.5.

```
glm.pred <- rep("Down", 1250)
glm.pred[glm.probs > .5] = "Up"
glm.pred
```

```
##  [1] "Up"    "Down"  "Down"  "Up"    "Up"    "Up"    "Down"  "Up"    "Up"    "Down"
## [11] "Down"  "Up"    "Up"    "Down"  "Down"  "Up"    "Up"    "Up"    "Up"    "Down"
## [21] "Up"    "Up"    "Up"    "Down"  "Up"    "Up"    "Down"  "Up"    "Up"    "Up"
## [31] "Up"    "Up"    "Down"  "Down"  "Up"    "Up"    "Up"    "Down"  "Down"  "Down"
## [41] "Down"  "Up"    "Up"    "Up"    "Up"    "Up"    "Up"    "Down"  "Up"    "Up"
## [51] "Up"    "Down"  "Down"  "Down"  "Up"    "Up"    "Down"  "Up"    "Up"    "Up"
## [61] "Down"  "Down"  "Up"    "Down"  "Down"  "Down"  "Down"  "Up"    "Down"  "Down"
## [71] "Up"    "Up"    "Up"    "Down"  "Down"  "Down"  "Up"    "Down"  "Up"    "Up"
## [81] "Down"  "Down"  "Up"    "Up"    "Up"    "Up"    "Down"  "Down"  "Down"  "Down"
## [91] "Up"    "Down"  "Up"    "Up"    "Up"    "Up"    "Up"    "Up"    "Up"    "Down"
## [ reached 'max' / getOption("max.print") -- omitted 1150 entries ]
```

The first command creates a vector of 1,250 `Down` elements. The second line transforms to `Up` all of the elements for which the predicted probability of a market increase exceeds 0.5. Given these predictions, the `table()` function can be used to produce a confusion matrix in order to determine how many observations were correctly or incorrectly classified.

```
tab=table(glm.pred, Direction)
tab
```

```
##      Direction
## glm.pred Down  Up
##      Down  145 141
##      Up    457 507
```

```
good.score=(tab[1,1]+tab[2,2])/sum(tab)
good.score
```

```
## [1] 0.5216
```

```
mean(glm.pred == Direction)
```

```
## [1] 0.5216
```

```
mean(Direction=="Up")
```

```
## [1] 0.5184
```

The diagonal elements of the confusion matrix indicate correct predictions, while the off-diagonals represent incorrect predictions. Hence our model correctly predicted that the market would go up on 507 days and that it would go down on 145 days, for a total of $507 + 145 = 652$ correct predictions. The `mean()` function can be used to compute the fraction of days for which the prediction was correct. In this case, logistic regression correctly predicted the movement of the market 52.2% of the time.

At first glance, it appears that the logistic regression model is working a little better than random guessing. However, this result is misleading because we trained and tested the model on the same set of 1,250 observations. In other words, $100\% - 52.2\% = 47.8\%$, is the *training* error rate. As we have seen previously, the training error rate is often overly optimistic—it tends to underestimate the test error rate. In order to better assess the accuracy of the logistic regression model in this setting, we can fit the model using part of the data, and then examine how well it predicts the *held out* data. This will yield a more realistic error rate, in the sense that in practice we will be interested in our model's performance not on the data that we used to fit the model, but rather on days in the future for which the market's movements are unknown.

To implement this strategy, we will first create a vector corresponding to the observations from 2001 through 2004. We will then use this vector to create a held out data set of observations from 2005.

```
train <- (Year < 2005)
#train
#train[1239]
Smarket.2005 <- Smarket[!train, ]
dim(Smarket.2005)

## [1] 252    9

Direction.2005 <- Direction[!train]
#Direction.2005
```

The object `train` is a vector of 1250 elements, corresponding to the observations in our data set. The elements of the vector that correspond to observations that occurred before 2005 are set to `TRUE`, whereas those that correspond to observations in 2005 are set to `FALSE`. The object `train` is a *Boolean* vector, since its elements are `TRUE` and `FALSE`. Boolean vectors can be used to obtain a subset of the rows or columns of a matrix. For instance, the command `Smarket[train, ]` would pick out a submatrix of the stock market data set, corresponding only to the dates before 2005, since those are the ones for which the elements of `train` are `TRUE`. The `!` symbol can be used to reverse all of the elements of a Boolean vector. That is, `!train` is a vector similar to `train`, except that the elements that are `TRUE` in `train` get swapped to `FALSE` in `!train`, and the elements that are `FALSE` in `train` get swapped to `TRUE` in `!train`. Therefore, `Smarket[!train, ]` yields a submatrix of the stock market data containing only the observations for which `train` is `FALSE`—that is, the observations with dates in 2005. The output above indicates that there are 252 such observations.

We now fit a logistic regression model using only the subset of the observations that correspond to dates before 2005, using the `subset` argument. We then obtain predicted probabilities of the stock market going up for each of the days in our test set—that is, for the days in 2005.

```
glm.fits <- glm(
  Direction ~ Lag1 + Lag2 + Lag3 + Lag4 + Lag5 + Volume,
  data = Smarket, family = binomial, subset = train
)
glm.probs <- predict(glm.fits, Smarket.2005,
  type = "response")
glm.probs
```

```
##      999      1000      1001      1002      1003      1004      1005      1006
## 0.5282195 0.5156688 0.5226521 0.5138543 0.4983345 0.5010912 0.5027703 0.5095680
##      1007      1008      1009      1010      1011      1012      1013      1014
## 0.5040112 0.5106408 0.5101183 0.4811653 0.5052950 0.5236316 0.5168364 0.5125333
##      1015      1016      1017      1018      1019      1020      1021      1022
## 0.4982179 0.4882768 0.4960135 0.5051879 0.4910689 0.4789755 0.4912577 0.5056236
##      1023      1024      1025      1026      1027      1028      1029      1030
## 0.4889100 0.4967660 0.5084460 0.5168250 0.5073168 0.4868397 0.5007467 0.5009795
##      1031      1032      1033      1034      1035      1036      1037      1038
## 0.5012692 0.5123720 0.5113280 0.5141906 0.5120219 0.4848925 0.4804829 0.4974951
```

```
##      1039      1040      1041      1042      1043      1044      1045      1046
## 0.5001878 0.4956115 0.4997080 0.4883755 0.4893983 0.5088590 0.5195232 0.5088729
##      1047      1048      1049      1050      1051      1052      1053      1054
## 0.5109177 0.5062938 0.5075737 0.5188956 0.5089908 0.4750034 0.5019550 0.5068664
##      1055      1056      1057      1058      1059      1060      1061      1062
## 0.4920794 0.4956677 0.4938379 0.4917051 0.4771327 0.4677842 0.4953634 0.4939976
##      1063      1064      1065      1066      1067      1068      1069      1070
## 0.4815092 0.4868789 0.4838154 0.5048192 0.5139716 0.4819605 0.4958307 0.5082474
##      1071      1072      1073      1074      1075      1076      1077      1078
## 0.5024951 0.4953088 0.4707835 0.4934152 0.4724764 0.4731641 0.4924263 0.4962637
##      1079      1080      1081      1082      1083      1084      1085      1086
## 0.4884338 0.4957835 0.4744252 0.4699449 0.4806214 0.4688377 0.4783015 0.5042272
##      1087      1088      1089      1090      1091      1092      1093      1094
## 0.4880608 0.4981440 0.5023063 0.4978148 0.5002376 0.4854172 0.4691275 0.4626001
##      1095      1096      1097      1098
## 0.4817722 0.4989457 0.4971733 0.4937147
## [ reached 'max' / getOption("max.print") -- omitted 152 entries ]
```

Notice that we have trained and tested our model on two completely separate data sets: training was performed using only the dates before 2005, and testing was performed using only the dates in 2005. Finally, we compute the predictions for 2005 and compare them to the actual movements of the market over that time period.

```
glm.pred <- rep("Down", 252)
glm.pred[glm.probs > .5] <- "Up"
table(glm.pred, Direction.2005)
```

```
##      Direction.2005
## glm.pred Down Up
##      Down   77 97
##      Up    34 44
```

```
mean(glm.pred == Direction.2005)
```

```
## [1] 0.4801587
```

```
mean(glm.pred != Direction.2005)
```

```
## [1] 0.5198413
```

The `!=` notation means *not equal to*, and so the last command computes the **test set error rate**. The results are rather disappointing: the test error rate is 52%, which is worse than random guessing! Of course this result is not all that surprising, given that one would not generally expect to be able to use previous days' returns to predict future market performance. (After all, if it were possible to do so, then the authors of this book would be out striking it rich rather than writing a statistics textbook.)

We recall that the logistic regression model had very underwhelming *p*-values associated with all of the predictors, and that the smallest *p*-value, though not very small, corresponded to `lagone`. Perhaps by removing the variables that appear not to be helpful in predicting `direction`, we can obtain a more effective model. After all, using predictors that have no relationship with the response tends to cause a deterioration in the test error rate (since such predictors cause an increase in variance without a corresponding decrease in bias), and so removing such predictors may in turn yield an improvement. Below we have refit the logistic regression using just `lagone` and `lagtwo`, which seemed to have the highest predictive power in the original logistic regression model.

```
glm.fits <- glm(Direction ~ Lag1 + Lag2, data = Smarket,
  family = binomial, subset = train)
glm.probs <- predict(glm.fits, Smarket.2005,
```

```

    type = "response")
glm.pred <- rep("Down", 252)
glm.pred[glm.probs > .5] <- "Up"
(tab=table(glm.pred, Direction.2005))

```

```

##           Direction.2005
## glm.pred Down  Up
##      Down   35  35
##      Up    76 106

```

```

mean(glm.pred == Direction.2005)

```

```

## [1] 0.5595238

```

```

(correct.ups=tab[2,2]/ (tab[2,1] + tab[2,2]))

```

```

## [1] 0.5824176

```

Now the results appear to be a little better: 56% of the daily movements have been correctly predicted. It is worth noting that in this case, a much simpler strategy of predicting that the market will increase every day will also be correct 56% of the time! Hence, in terms of overall error rate, the logistic regression method is no better than the naive approach. However, the confusion matrix shows that on days when logistic regression predicts an increase in the market, it has a 58% accuracy rate. This suggests a possible trading strategy of buying on days when the model predicts an increasing market, and avoiding trades on days when a decrease is predicted. Of course one would need to investigate more carefully whether this small improvement was real or just due to random chance.

Suppose that we want to predict the returns associated with particular values of `lagone` and `lagtwo`. In particular, we want to predict `direction` on a day when `lagone` and `lagtwo` equal 1.2 and 1.1, respectively, and on a day when they equal 1.5 and -0.8 . We do this using the `predict()` function.

```

predict(glm.fits,
  newdata =
    data.frame(Lag1 = c(1.2, 1.5), Lag2 = c(1.1, -0.8)),
  type = "response"
)

```

```

##           1           2
## 0.4791462 0.4960939

```